

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине
«Введение в нереляционные системы управления базами данных»
Тема: Хранилище изображений с CRUD

Студент гр. 3388

Ижболдин А.В.

Студент гр. 3388

Еникеев А.А.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студент Ижболдин А.В.

Студент Еникеев А.А.

Группа 3388

Тема работы: 04. Хранилище изображений с CRUD

Исходные данные:

Задача - реализуйте веб-приложение с REST интерфейсом для CRUD изображений. В веб-интерфейсе реализуйте возможность поиска изображений по метаданным, датам, данным самого изображения. Храните все логи .

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 12.05.2026

Дата защиты реферата: 12.05.2026

Студент		Ижболдин А.В.
Студент		Еникеев. А.А.
Преподаватель		Заславский М.М.

АННОТАЦИЯ

В данной курсовой работе рассматривается процесс проектирования и реализации веб-приложения для управления хранилищем изображений с использованием архитектурного стиля REST. Основная цель работы заключается в создании масштабируемой системы, обеспечивающей выполнение операций CRUD (создание, чтение, обновление, удаление) над графическими объектами. В ходе разработки использовались современные нереляционные системы управления базами данных для эффективного хранения метаданных и самих файлов изображений.

Особое внимание уделено реализации функционала расширенного поиска, позволяющего пользователю находить изображения по различным критериям: метаданным (формат, разрешение, автор), временным интервалам (даты загрузки или создания) и атрибутам самого контента. Программный комплекс также включает в себя систему логирования всех транзакций и действий пользователей для обеспечения аудита и мониторинга состояния сервиса. Результатом работы является полнофункциональное веб-приложение, соответствующее заданным качественным требованиям.

SUMMARY

This course project focuses on the development of a web application for image storage management utilizing a RESTful interface. The project implements a complete set of CRUD operations for image data, leveraging non-relational database technologies for optimized storage of both files and associated metadata.

A key feature of the application is the advanced search engine, which enables users to filter images based on technical metadata, specific dates, and internal image properties. Furthermore, a comprehensive logging system was integrated to track all system events and user activities. The final result is a robust web service that demonstrates the practical application of NoSQL databases in modern web development.

СОДЕРЖАНИЕ

	Введение	6
1.	Выполнение задания	9
1.1.	Согласована и сформулирована тема ИДЗ	9
1.2.	Сценарии использования	9
1.2.1.	Макет пользовательского интерфейса:	10
1.2.2.	Сценарии использования.	11
1.2.3.	Преобладающий тип операций.	21
1.3.	Модель данных	21
1.3.1.	Нереляционная модель данных (для вашей СУБД).	21
1.3.2.	Аналог модели данных для SQL СУБД	33
1.3.3.	Сравнение моделей	47
1.4.	Разработанное приложение	50
	Заключение	54
	ПРИЛОЖЕНИЕ А. ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ	55
	ПРИЛОЖЕНИЕ Б	56
	ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ	

ВВЕДЕНИЕ

Актуальность решаемой проблемы

В условиях экспонентального роста объемов цифрового контента, в частности, изображений, критически важной становится проблема их эффективного и масштабируемого хранения, а также организации быстрого и гибкого доступа к ним. Традиционные реляционные подходы часто сталкиваются с ограничениями при работе с неструктурированными данными и переменчивыми схемами метаданных, что усложняет реализацию функций расширенного поиска. Актуальность проекта обусловлена необходимостью разработки современного, высоконагруженного решения, которое, используя преимущества нереляционных СУБД (MongoDB), обеспечит не только базовые операции CRUD, но и сложный поиск по метаданным, датам и содержимому. Дополнительно, интеграция системы логирования и многоуровневой авторизации является требованием современных веб-сервисов для обеспечения безопасности, аудита действий пользователей и контроля доступа. Ну, а также этот проект позволит изучить и освоить нереляционные СУБД.

Постановка задачи

Основной целью является проектирование и разработка масштабируемого веб-приложения для создания хранилища изображений с полной реализацией операций CRUD (создание, чтение, обновление, удаление), расширенным функционалом поиска и системой контроля доступа, используя нереляционную базу данных.

Для достижения поставленной цели необходимо решить следующие задачи:

- Разработать RESTful API для управления контентом и метаданными изображений, включая автоматическое определение формата, размера и разрешения файла.
- Реализовать клиентскую часть (SPA) для взаимодействия с API, обеспечивающую загрузку, отображение, поиск по текстовым полям и фильтрацию изображений (в том числе по лайкам).

- Внедрить социальные функции (систему лайков с подсчетом общего количества) и систему логирования действий пользователей.
- Создать многоуровневую систему авторизации и прав доступа (автор, администратор, неавторизованный пользователь) с механизмами блокировки публикации и ограничения активности.
- Обеспечить контейнеризацию всех компонентов (backend, frontend, DB, Nginx) с использованием Docker Compose для упрощения развертывания и разработки.

Предлагаемое решение

- **Архитектура и стек:** Использование клиент-серверной архитектуры (SPA) на базе современного веб-стека. Backend реализован на **Python** с **Django/DRF** и **Pillow**, а Frontend — на **Vue.js 3/TypeScript** с управлением состоянием через **Pinia** и стилизацией через **Tailwind CSS**.
- **Хранение данных:** Применение нереляционной СУБД **MongoDB** с интеграцией через **django-mongodb-backend** для эффективного управления метаданными.
- **Развертывание:** Декларативное развертывание всех сервисов осуществляется с помощью **Docker Compose**, а веб-сервер **Nginx** используется как обратный прокси.
- **Безопасность:** Реализация троттлинга для защиты от спама и использование встроенных механизмов Django/DRF для контроля прав доступа.

Качественные требования к решению

- **Безопасность:**
 - Должна быть реализована многоуровневая система авторизации с разграничением прав доступа (Администратор, Автор, Неавторизованный пользователь).
 - Необходимо внедрение механизмов защиты от DoS/DDoS атак, включая троттлинг запросов к API.
- **Надежность и аудит:**

- Все действия пользователей и транзакции с данными должны быть зафиксированы в системе логирования для целей аудита.
- Данные метаданных должны быть целостными и всегда соответствовать сохраненным файлам.
- **Сопровождаемость:**
 - Все компоненты приложения должны быть контейнеризированы с помощью Docker Compose для обеспечения простоты развертывания и переноса между средами.
 - Код должен соответствовать принятым стандартам.
- **Удобство использования (Usability):**
 - Клиентский интерфейс (SPA) должен быть интуитивно понятным, обеспечивать быструю загрузку страниц и корректное отображение изображений на различных устройствах.

1. ВЫПОЛНЕНИЕ ЗАДАНИЯ

1.1. **Согласована и сформулирована тема ИДЗ**

В рамках первого этапа работы **«Согласована и сформулирована тема ИДЗ»** были выполнены следующие действия:

Согласование темы и инструментов Была выбрана и согласована тема курсовой работы: «Хранилище изображений с CRUD». Было предложено видение устройства приложения и озвучены используемые инструменты (язык программирования и СУБД).

Получение доступа к репозиторию Был получен и проверен доступ к репозиторию.

Подготовка минимального примера Для исключения проблем совместимости был подготовлен минимально работоспособный пример (скрипт для командной строки) для выбранной пары «База данных + Язык программирования». Этот скрипт подключился к БД, записал и прочитал данные.

Демонстрация и загрузка кода Работоспособность скрипта была продемонстрирована в виде скринкаста, ссылка на который отправлена преподавателю. Код тестового приложения загружен в репозиторий, в ветку main, в каталог hello_world.

1.2. **Сценарии использования**

Результаты этого этапа работы были оформлены в виде отдельной страницы на вики-репозитории под названием «Макет и сценарий использования».

Содержание вики-страницы: На странице размещены формулировки основных сценариев использования приложения в текстовом виде, а также макет пользовательского интерфейса (большое изображение).

1.2.1. Макет пользовательского интерфейса:

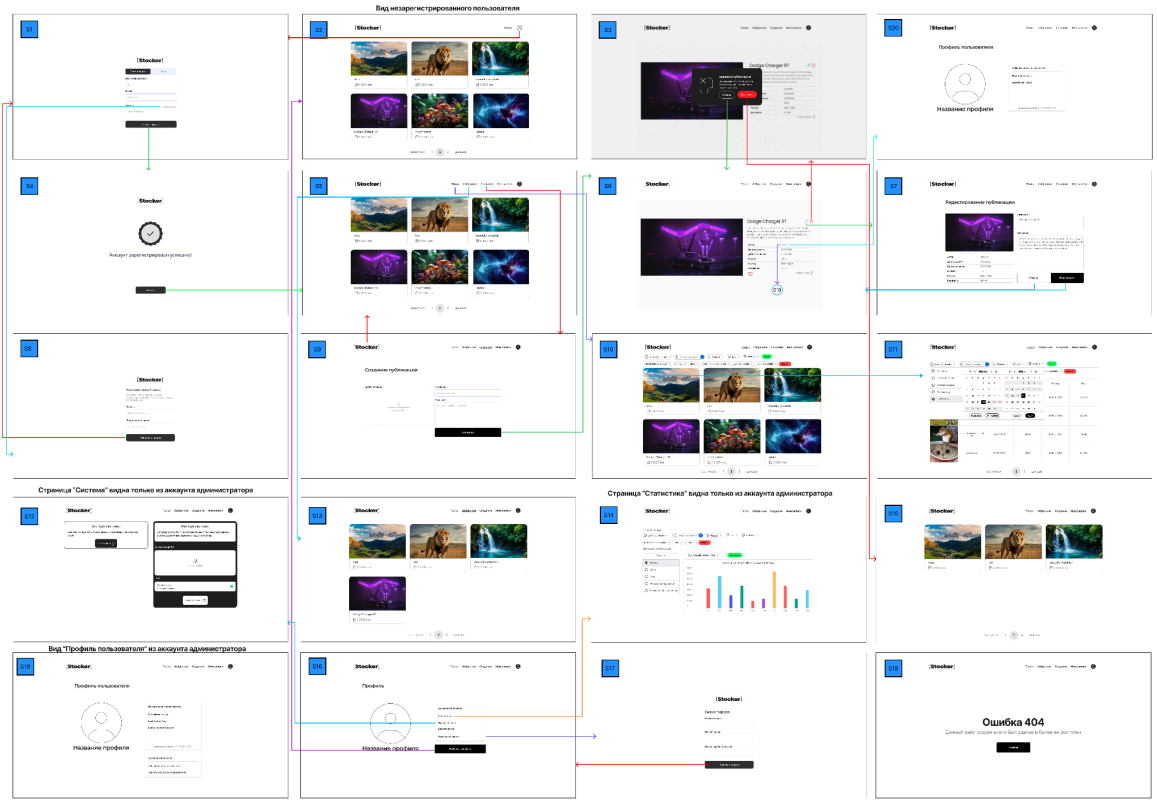


Рис №1. МАКЕТ

Макет представлен в виде графа, демонстрирующего переходы между экранами.

Файл макета загружен в репозиторий под именем `ui_mockup.png` и вставлен на вики-страницу как изображение.

Каждый экран на макете имеет уникальное название или номер, дублирование экранов отсутствует.

Все надписи и элементы на макете различимы и читаемы.

Функциональные элементы макета: Макет включает в себя элементы управления для: Массового импорта/экспорта данных. Построения кастомизируемой статистики, требования к которой описаны в задании.

Сценарии для демонстрации в прототипе:

- Регистрация новых пользователей
- Загрузка изображения
- Редактирование метаданных
- Удаление изображения

- Поиск и фильтрация

1.2.2. Сценарии использования.

Сценарий 1: Регистрация нового пользователя (Экран S1 -> S4)

Предусловие: Пользователь не авторизован и находится на странице входа/регистрации (Экран S1).

Основной сценарий:

Пользователь переключается на вкладку «Регистрация».

Пользователь вводит имя пользователя в поле "Имя пользователя" (Тип: текстовая строка, латиница/цифры).

Пользователь вводит свой email в поле "Email" (Тип: текстовая строка, формат email).

Пользователь вводит пароль в поле "Пароль" и повторяет его в следующем поле (Тип: скрытая текстовая строка).

Пользователь нажимает кнопку «Создать аккаунт».

Система проверяет данные, создает запись в БД и показывает экран успешной регистрации (S4).

Пользователь нажимает кнопку «Начать» на экране S4 и перенаправляется на главную страницу авторизованного пользователя (S5).

Нежелательное завершение (Ошибка):

Ошибка валидации: Введенный email уже существует в базе или пароли не совпадают. Система подсвечивает поля красным и выводит текстовое сообщение об ошибке. Регистрация прерывается.

Ошибка соединения: При нажатии на кнопку пропадает интернет-соединение. Система выводит уведомление "Ошибка сети. Попробуйте позже".

Отмена операции:

Пользователь до нажатия кнопки "Создать аккаунт" переключается на вкладку "Вход" или закрывает вкладку браузера.

Результат: Аккаунт создан, в БД появилась новая запись пользователя, создана запись в коллекции logs о регистрации. Пользователь авторизован в системе.

Сценарий 2: Загрузка нового изображения / Create (Экран S5 -> S9)

Предусловие: Пользователь авторизован и находится на любой странице приложения.

Основной сценарий:

Пользователь нажимает на пункт меню «Создание» в верхней навигационной панели.

Система открывает страницу создания публикации (S9).

Пользователь нажимает на область "Добавьте файл" и выбирает файл через системное окно или перетаскивает файл (Drag&Drop) в эту область (Тип: бинарный файл, форматы: .jpg, .png, .bmp, ограничение веса: до 10 МБ).

Пользователь вводит название в поле "Название" (Тип: текстовая строка).

Пользователь вводит описание в поле "Описание" (Тип: текстовая строка многострочная).

Пользователь нажимает кнопку «Отправить».

Система загружает файл, автоматически высчитывает метаданные (разрешение, формат, вес, дата создания) и перенаправляет пользователя на страницу просмотра этого изображения.

Нежелательное завершение (Ошибка):

Неверный формат/размер: Пользователь загружает PDF или файл весом 50 МБ. Система моментально сбрасывает файл и показывает ошибку "Неподдерживаемый формат или файл слишком велик".

Ошибка соединения: Во время загрузки тяжелого файла обрывается связь. Прогресс-бар зависает, система выдает таймаут-ошибку "Не удалось загрузить файл на сервер".

Отмена операции:

На этапе ввода данных (до нажатия "Отправить") пользователь нажимает на пункт меню "Поиск" или "Главная". Данные сбрасываются, сохранение не происходит.

Результат: Изображение физически сохранено в хранилище, в MongoDB создана запись со всеми ручными и автоматическими метаданными. В коллекции logs зафиксировано действие image_uploaded.

Сценарий 3: Редактирование метаданных публикации / Update (Экран S6 -> S7)

Предусловие: Пользователь авторизован, находится на странице просмотра изображения (S6), созданного им самим (является автором).

Основной сценарий:

Пользователь нажимает на иконку "Карандаш" (Редактировать) рядом с названием публикации.

Система открывает страницу редактирования (S7). Поля "Название" и "Описание" предзаполнены текущими данными. Технические данные (Разрешение, Формат и т.д.) отображаются только для чтения.

Пользователь кликает в поле "Название" и изменяет текст (Тип: текстовая строка).

Пользователь кликает в поле "Описание" и дополняет текст (Тип: текстовая строка многострочная).

Пользователь нажимает кнопку «Подтвердить».

Система сохраняет изменения и возвращает пользователя на обновленную страницу просмотра изображения (S6).

Нежелательное завершение (Ошибка):

Потеря прав сессии: Во время редактирования истек токен сессии. При нажатии "Подтвердить" система выдает ошибку 401 Unauthorized и перекидывает на экран авторизации (S1).

Ошибка БД: Сбой записи в MongoDB. Выводится тост-уведомление "Не удалось сохранить изменения".

Отмена операции:

Пользователь нажимает кнопку «Отмена» (белая кнопка рядом с "Подтвердить"). Система возвращает пользователя на экран S6 без применения изменений.

Результат: Метаданные изображения обновлены в БД. В logs зафиксировано metadata_updated.

Сценарий 4: Удаление публикации / Delete (Экран S6 -> S3)

Предусловие: Пользователь авторизован, находится на странице просмотра своего изображения (S6).

Основной сценарий:

Пользователь нажимает на иконку "Корзина" (Удалить) рядом с названием публикации.

Система открывает модальное окно (S3) с текстом предупреждения "Вы уверены, что хотите удалить эту публикацию? Это действие нельзя отменить".

Пользователь нажимает красную кнопку «Да, удалить».

Система удаляет файл с сервера и запись из БД, после чего перенаправляет пользователя на главную страницу (S5).

Нежелательное завершение (Ошибка):

Конфликт удаления: Изображение было удалено модератором или сервером миллисекундой ранее. Сервер возвращает 404 Not Found, система выводит сообщение "Публикация уже не существует".

Отмена операции:

Пользователь в модальном окне (S3) нажимает серую кнопку «Отмена» или кликает мышью в затемненную область вне модального окна. Окно закрывается, пользователь остается на странице S6.

Результат: Файл удален, документ в MongoDB удален. В logs зафиксировано image_deleted.

Сценарий 5: Сложный поиск и фильтрация / Read & Search (Экран S10, S11)

Предусловие: Пользователь авторизован и перешел в раздел табличного представления/поиска (S10).

Основной сценарий:

Пользователь кликает в поле "Введите название" и вводит искомое слово, например "animals" (Тип: текстовая строка).

Пользователь нажимает на фильтр "Формат" и выбирает чекбокс "JPEG" из выпадающего списка (Тип: массив строк/Enum).

Пользователь нажимает на фильтр "Дата публикации" (S11), открывается календарь. Пользователь выбирает начальную и конечную дату (Тип: массив из двух дат DateTime). Нажимает "Apply".

Система динамически (или по нажатию кнопки поиска) фильтрует таблицу ниже, оставляя только те записи, которые удовлетворяют всем выбранным условиям (название содержит "animals", формат JPEG, загружено в выбранный период).

Нежелательное завершение (Ошибка):

Ошибка соединения: При отправке параметров фильтрации на сервер интернет пропадает. Таблица переходит в состояние вечной загрузки (скелетон или спиннер), затем выдает ошибку "Не удалось загрузить данные поиска".

Слишком тяжелый запрос: БД не отвечает по таймауту (маловероятно для синтетики, но возможно). Вывод сообщения об ошибке сервера.

Отмена операции:

Пользователь нажимает красную кнопку "Clear All" (Очистить все). Все введенные текстовые фильтры стираются, чекбоксы сбрасываются. Таблица возвращается к отображению всех существующих записей.

Результат: Пользователь видит нужную выборку изображений. В коллекции logs зафиксировано действие search_executed с параметрами поискового запроса пользователя (для аналитики).

Сценарий 6 (Админ): Просмотр статистики системы (Экран S14)

Предусловие: Пользователь авторизован под аккаунтом с ролью Administrator. Пользователь кликнул на иконку профиля и выбрал "Статистика" (как показано на S16).

Основной сценарий:

Пользователь попадает на страницу "Статистика" (S14).

Пользователь может выбрать параметры для осей X, Y для построения графиков.

Пользователь нажимает зеленую кнопку "Построить".

Система запрашивает агрегированные данные из коллекции и перерисовывает столбчатую диаграмму, отображая статистику загрузок по дням.

Нежелательное завершение (Ошибка):

Нет данных: За выбранный период в системе не было активности. График отображается пустым с надписью "Нет данных для отображения за этот период".

Отмена операции:

До нажатия кнопки "Построить" пользователь может передумать и выбрать другой фильтр, либо уйти с раздела через верхнее меню.

Результат: Администратор получает визуализированную аналитику по использованию хранилища. Запись о просмотре логов администратором также может быть добавлена в logs.

Сценарий 7: Экспорт базы данных системы (Экран S12, левая часть)

Предусловие: Пользователь авторизован под аккаунтом с ролью Administrator. Пользователь открыл меню профиля, нажал «Импорт/Экспорт» (S16) и находится на странице системы (S12).

Основной сценарий:

Пользователь в блоке «Экспорт системы» нажимает кнопку «Скачать файл».

Система создает изолированную временную директорию сборки, обращается к БД (MongoDB) и собирает все текущие записи изображений, метаданные и логи.

Система полностью копирует директорию с пользовательскими медиафайлами во временную папку сборки.

Система упаковывает подготовленные файлы (дампы БД и медиафайлы) в единый архив с именем формата `stocker_backup_YYYYMMDD_NHMMSS.zip` (Тип: бинарная передача файла архива, строго формат .zip).

Система инициирует скачивание файла средствами браузера (появляется стандартное окно сохранения ZIP-архива на устройство пользователя).

Пользователь сохраняет файл на свой жесткий диск.

Нежелательное завершение (Ошибка):

Таймаут БД: База данных слишком велика, и процесс генерации файла занимает больше времени, чем разрешено настройками сервера. Сервер обрывает соединение, система выводит тост-уведомление "Ошибка экспорта: превышено время ожидания сервера".

Ошибка соединения: В момент скачивания сгенерированного ZIP-архива обрывается интернет пользователя. Загрузка в браузере помечается как "Ошибка сети".

Отмена операции:

Когда браузер показывает системное окно выбора места для сохранения файла, пользователь нажимает кнопку «Отмена» в окне ОС. Файл не сохраняется на устройство пользователя.

Результат: Пользователь получил полную резервную копию базы данных приложения в виде ZIP-архива на свое локальное устройство. В коллекции logs зафиксировано действие `database_exported` с указанием времени и id администратора.

Сценарий 8: Импорт (восстановление) базы данных (Экран S12, правая часть)

Предусловие: Пользователь авторизован под аккаунтом с ролью Administrator. Пользователь находится на странице системы (S12). У пользователя на локальном устройстве есть корректный ZIP-архив резервной копии системы (содержащий дамп БД `db.json` и папку с медиафайлами `uploads`).

Основной сценарий:

В блоке «Импорт системы» пользователь кликает на область загрузки файла "Click to upload" или перетаскивает в неё файл с компьютера (Тип: бинарная передача файла, строго формат .zip).

Система валидирует расширение файла и отображает его имя (например, stocker_backup_20260213_103000.zip) внутри области загрузки.

Пользователь нажимает черную кнопку «Импортировать».

Система отправляет архив на бэкенд.

Бэкенд сохраняет архив и проводит проверки безопасности (отсутствие небезопасных путей и превышения лимита распаковки).

Система создает резервную копию текущего состояния базы данных и папки медиафайлов для возможности отката.

Бэкенд полностью очищает текущие таблицы целевых приложений в базе данных и заменяет папку медиафайлов содержимым из архива, после чего загружает новые записи из распакованного db.json.

Система выводит сообщение об успешном импорте и принудительно обновляет страницу (или перекидывает на главную страницу S5) для отображения нового состояния базы.

Нежелательное завершение (Ошибка):

Невалидный файл или формат: Пользователь загрузил файл с расширением, отличным от .zip, или архив не содержит внутри обязательного файла db.json. Бэкенд возвращает ошибку HTTP 400 ("Необходимо загрузить корректный .zip архив" или "В архиве отсутствует файл db.json"). Текущая система остается без изменений.

Поврежденные данные или сбой схемы БД: Архив распаковался, но структура внутри db.json нарушена (пустой файл, синтаксическая ошибка JSON или конфликт внешних ключей при загрузке). Бэкенд прерывает операцию, автоматически выполняет полный откат базы данных и медиафайлов к исходному состоянию из резервной копии и возвращает HTTP 500.

Ошибка соединения во время загрузки: Пользователь нажал «Импортировать», но во время передачи файла на сервер пропал интернет. Система выдает ошибку. Данные на сервере не затрагиваются.

Отмена операции:

Пользователь выбрал файл (шаг 2), но передумал и до нажатия кнопки «Импортировать» ушел на другую страницу (например, нажал "Поиск" в шапке). Операция прерывается, выбранный файл сбрасывается из памяти браузера, изменений на сервере не происходит.

Результат: Текущая база данных и пользовательские медиафайлы полностью перезаписаны данными из загруженного ZIP-архива. В новые логи (или поверх импортированных) записывается системное событие `database_imported`.

Сценарий 9: Изменение пароля в профиле (Экран S16 -> S17)

Предусловие: Пользователь авторизован в системе. Пользователь нажал на иконку профиля в шапке, выбрал в выпадающем меню пункт «Изменение пароля» (S16) и находится на странице смены пароля (S17).

Основной сценарий:

Пользователь кликает в поле «Старый пароль» и вводит свой текущий пароль (Тип: скрытая текстовая строка).

Пользователь кликает в поле «Новый пароль» и вводит желаемый новый пароль, отвечающий требованиям безопасности (например, минимум 8 символов) (Тип: скрытая текстовая строка).

Пользователь кликает в поле «Новый пароль (еще раз)» и повторно вводит новый пароль (Тип: скрытая текстовая строка).

Пользователь нажимает черную кнопку «Сменить пароль».

Система проверяет совпадение старого пароля с базой данных и идентичность новых паролей.

Система обновляет хэш пароля в базе данных (MongoDB).

Система выводит уведомление «Пароль успешно изменен» и перенаправляет пользователя в профиль (S16).

Нежелательное завершение (Ошибка):

Неверный старый пароль: Пользователь опечатался в поле «Старый пароль». При нажатии кнопки система подсвечивает поле красным и выводит ошибку «Текущий пароль введен неверно». Обновление не происходит.

Пароли не совпадают: Значения в полях нового пароля различаются. Система блокирует отправку и выводит текстовую ошибку «Новые пароли не совпадают».

Ошибка соединения: При отправке формы пропадает интернет. Браузер зависает в ожидании ответа, затем система выводит таймаут-ошибку "Нет связи с сервером. Попробуйте позже".

Отмена операции:

Пользователь ввел данные, но передумал и до нажатия кнопки «Сменить пароль» перешел в другой раздел через верхнее меню (например, кликнул «Поиск»). Введенные данные стираются, пароль остается прежним.

Результат: Пароль пользователя изменен в БД. Прежний пароль больше не действителен. В коллекции logs зафиксировано системное событие password_changed.

Сценарий 10: Восстановление / Установка нового пароля (Экран S1 -> S8)

Предусловие: Пользователь не авторизован. Пользователь забыл пароль, нажал ссылку «Забыли пароль?» на экране входа (S1), получил письмо на свой Email и перешел по уникальной одноразовой ссылке восстановления из письма. Пользователь находится на странице установки нового пароля (S8).

Основной сценарий:

Пользователь кликает в поле «Пароль» и вводит новый пароль (Тип: скрытая текстовая строка).

Пользователь кликает в поле «Подтвердите пароль» и вводит тот же пароль повторно (Тип: скрытая текстовая строка).

Пользователь нажимает черную кнопку «Обновить пароль».

Система валидирует токен восстановления (защитый в URL), проверяет совпадение паролей.

Система сохраняет новый хэш пароля в БД и аннулирует токен восстановления.

Система автоматически авторизует пользователя и перенаправляет его на страницу входа с уведомлением «Пароль успешно обновлен».

Нежелательное завершение (Ошибка):

Токен устарел или недействителен: Пользователь перешел по ссылке из письма суточной давности или ссылка уже была использована. При нажатии «Обновить пароль» система отклоняет запрос, выдает ошибку 403 Forbidden с текстом «Ссылка недействительна или устарела» и перенаправляет на экран входа (S1).

Ошибка соединения: При нажатии кнопки пропадает связь. Действие не завершается, система выводит стандартную ошибку сети. (При восстановлении связи пользователю нужно будет нажать кнопку снова, токен при этом останется валидным, так как операция на сервере не была завершена).

Отмена операции:

Пользователь перешел по ссылке на экран S8, но вспомнил старый пароль. Он просто закрывает вкладку браузера или нажимает кнопку «Назад» в браузере. Токен восстановления остается активным до истечения своего срока жизни (time-to-live), пароль в БД не меняется.

Результат: Пользователь восстановил доступ к аккаунту с новым паролем. Старый пароль (если он был) затерт в БД. В коллекции logs зафиксировано событие password_recovered. Пользователь авторизован в сессии.

Сценарии к прототипу "Хранение и представление"

Мы считаем что данные сценарии являются базой на которой можно будет провести сравнение SQL и NoSQL моделей.

Регистрация новых пользователей

Загрузка изображения

Редактирование метаданных

Удаление изображения

Поиск и фильтрация

1.2.3. Преобладающий тип операций.

В проекте как и в любом практически хостинге преобладают READ операции над WRITE. Т.к. поиск и просмотр изображений является более частой операцией, чем создание изображений и редактирование метаданных.

1.3. Модель данных

Результаты этого этапа работы были оформлены в виде отдельной страницы на вики-репозитории под названием «Макет и сценарий использования»

1.3.1. Нереляционная модель данных (для вашей СУБД).

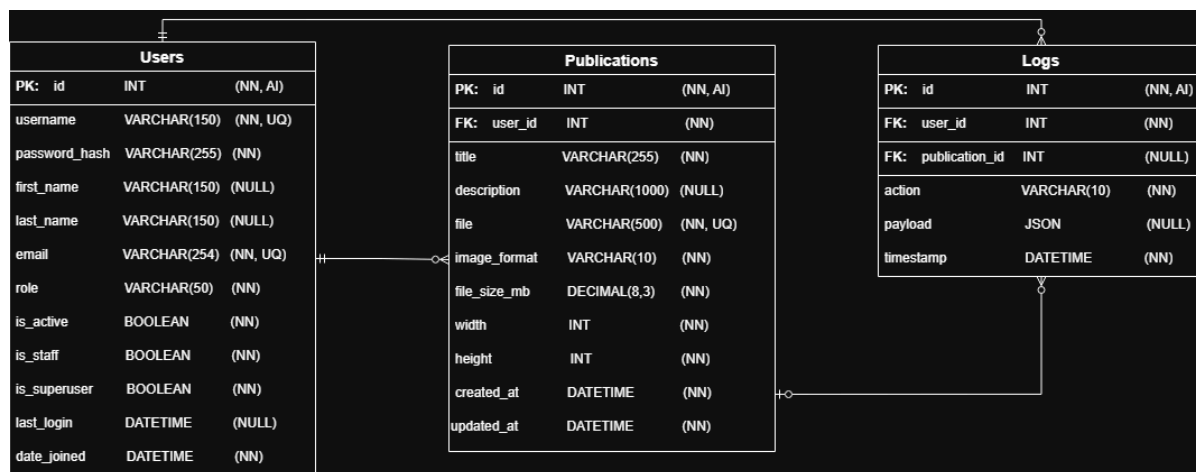


Рис №2. SQL модель

Описание таблиц, типов данных и сущностей

Таблица Users (Пользователи)

Что хранит: Учетные данные (логин, хэш пароля, email), системные роли (права администратора, статус персонала) и метки времени (дата регистрации, последний вход).

Для чего нужна: Отвечает за аутентификацию и авторизацию. Позволяет системе понимать, кто именно сделал запрос к API, какие у него права доступа

(может ли он редактировать чужие данные), позволяет привязать каждую загруженную картинку к её конкретному автору.

Атрибуты и типы данных:

id — Int32 (4 байта)

username, first_name, last_name, email, role — VARCHAR (2 байта на 1 символ)

password_hash — VARCHAR (2 байта на 1 символ)

is_superuser, is_staff, is_active — Boolean / TINYINT (1 байт)

last_login, date_joined — Datetime (5 байт)

Таблица Publications (Публикации / Изображения)

Что хранит: Метаданные загруженных файлов: пользовательские (название, описание), технические (ширина, высота, размер в МБ, формат), системные (даты создания и обновления), а также путь к файлу картинки на сервере. Хранит ссылку на создателя (user_id).

Для чего нужна: Таблица нужна для отображения галереи, фильтрации и поиска изображений по параметрам, а также для того, чтобы приложение знало, где именно лежит нужный файл, чтобы отдать его клиенту.

Атрибуты и типы данных:

id, user_id — Int32 (4 байта)

title, description, file, image_format — VARCHAR (2 байта на 1 символ)

width, height — Int32 (4 байта)

file_size_mb — Decimal (4 байта)

created_at, updated_at — Datetime (5 байт)

Таблица Logs (Журнал событий)

Что хранит: Хронологию всех значимых действий (CRUD). Фиксирует, кто совершил действие (`user_id`), тип операции (`action` — CREATE, UPDATE, DELETE, EXPORT) и точное время. Детали самого изменения (например, старое и новое название) сохраняются в поле `payload`.

Для чего нужна: Для контроля, безопасности и аналитики.

Атрибуты и типы данных:

`id`, `user_id`, `publication_id` — Int32 (4 байта)

`action` — VARCHAR (2 байта на 1 символ)

`payload` — JSON (2 байта на 1 символ сериализованной строки)

`timestamp` — Datetime (5 байт)

Оценка объема информации, хранимой в модели

(Типы данных и базовый вес полей описаны в предыдущем разделе. Для строк используется формула $V = 2 * N$, где N — среднее число символов).

Таблица "Users"

Фиксированные поля: `id` (4b) + `is_superuser` (1b) + `is_staff` (1b) + `is_active` (1b) + `last_login` (5b) + `date_joined` (5b) = 17b

Строковые поля:

`username` ($N \approx 10$): 20b

`password_hash` ($N \approx 88$): 176b

`password_hash` ($N \approx 88$): 176b

`email` ($N \approx 25$): 50b

`first_name`, `last_name` ($N \approx 10$ каждое): 40b

`role` ($N \approx 5$): 10b

Средний объем одной записи (V_u): 313b

Таблица "Publications"

Фиксированные поля: `id` (4b) + `author_id` (4b) + `width` (4b) + `height` (4b) + `file_size_mb` (4b) + `created_at` (5b) + `updated_at` (5b) = 30b

Строковые поля:

`title` ($N \approx 30$): 60b

description ($N \approx 100$): 200b

file ($N \approx 60$): 120b

image_format ($N \approx 4$): 8b

Средний объем одной записи (V_p): 418b

Таблица "Log"

Фиксированные поля: id (4b) + user_id (4b) + publication_id (4b) +
timestamp (5b) = 17b

Строковые поля:

action ($N \approx 6$): 12b

payload ($N \approx 50$): 100b

Средний объем одной записи (V_l): 129b

Оценка общего объема данных

Пусть N — общее количество загруженных публикаций (основная переменная, так как вокруг нее строится система).

Оценка количества пользователей (N_u): Большинство зарегистрированных пользователей загружают 1-3 картинки, и малая часть активно пополняет галерею. Примем в среднем 5 публикаций на 1 пользователя. Следовательно, количество пользователей: $N_u = 0.2 * N$.

Оценка количества логов (N_l): Система логов фиксирует CRUD операции. Жизненный цикл публикации включает 1 запись о загрузке, 1-2 записи о редактировании метаданных и многократные события чтения со стороны разных пользователей. Примем в среднем 10 логов на каждую публикацию. Следовательно, количество логов: $N_l = 10 * N$.

Объем данных для хранения базы из N публикаций:

$$V(N) = V_u * N_u + V_p * N + V_l * N_l$$

$$V(N) = 313 * (0.2 * N) + 418 * N + 129 * (10 * N)$$

$$V(N) = 62.6 * N + 418 * N + 1290 * N$$

$$V(N) = 1770.6 * N$$

Итоговая формула зависимости: $V(N) \approx 1771 * N$ (байт)

Пример для 100 000 загруженных публикаций: $V(100\ 000) = 1771 * 100\ 000 = 177\ 100\ 000\ \text{b} \approx 177.1\ \text{МБ}$

Избыточность данных

Оценка чистого объема данных (без учета технических ключей id и FK по 4 байта):

$$V_u\text{_pure}: 313 - 4 = 309\text{b}$$

$$V_p\text{_pure}: 418 - 8 = 410\text{b}$$

$$V_l\text{_pure}: 129 - 12 = 117\text{b}$$

Формула чистого объема: (с учетом выведенных ранее пропорций: $N_u = 0.2 * N$; $N_l = 10 * N$)

$$V\text{_pure}(N) = 309 * (0.2 * N) + 410 * N + 117 * (10 * N)$$

$$V\text{_pure}(N) = 1641.8 * N \text{ (байт)}$$

Расчет избыточности

Избыточность (R) вычисляется как отношение фактического объема ($V\text{_actual}$) к чистому ($V\text{_pure}$):

$$R = (1770.6 * N) / (1641.8 * N)$$

$$R \approx 1.078$$

Вывод: Избыточность реляционной модели составляет ~7.8%.

Направление роста модели при увеличении количества объектов каждой сущности

Рост реляционной модели направлен строго вертикально. Зависимость этого вертикального роста является линейной. При увеличении числа объектов одной сущности (и зафиксированном объеме остальных таблиц), размер модели увеличивается по следующим формулам:

Через пользователей (Users): $\text{modelSize_Users}(x) = 313 * x + C$ (где C — текущий объем публикаций и логов)

Через публикации (Publications): $\text{modelSize_Publications}(x) = 418 * x + C$ (где C — текущий объем пользователей и логов)

Через логи (Logs): $\text{modelSize_Logs}(x) = 129 * x + C$ (где C — текущий объем пользователей и публикаций)

Комплексный рост: Если рассматривать систему, когда объекты растут пропорционально друг другу: на 1 пользователя приходится 5 публикаций, а на 1 публикацию — 10 логов (таким образом, 1 пользователь генерирует 50 логов).
Общая зависимость размера БД от числа пользователей (x):

$$\text{modelSize_Total}(x) = 313 * x + 418 * (5 * x) + 129 * (50 * x)$$

$$\text{modelSize_Total}(x) = 8853 * x$$

Примеры данных

Users:

```
INSERT INTO Users (id, username, first_name, last_name, email,
password_hash, role, is_superuser, is_staff, is_active, last_login, date_joined)
VALUES
```

```
(1,'art_lover','Анна','Смирнова','anna.art@example.com','pbkdf2_sha256$260000$X
yZ123HashString...','user',FALSE,FALSE,TRUE,CURRENT_TIMESTAMP,CURRE
NT_TIMESTAMP);
```

Publications:

```
INSERT INTO Publications (id, user_id, title, description, file, image_format,
width, height, file_size_mb, created_at, updated_at)VALUES (101,1,'Киберпанк
город','Концепт-арт ночного города с неоновыми
вывесками.','media/photos/2026/4/cyber_city_01.png','PNG',3840,2160,8.4,CURRE
NT_TIMESTAMP,CURRENT_TIMESTAMP);
```

Logs:

```
INSERT INTO Logs (id, user_id, publication_id, action, payload, timestamp)
VALUES (5042,1,101,'CREATE',{'title': "Киберпанк город", "format":
"PNG"}',CURRENT_TIMESTAMP);
```

Примеры запросов

Сценарий 1: Регистрация нового пользователя

Текст запросов:

Проверка на уникальность email:

```
SELECT id FROM Users WHERE email = 'newuser@email.com';
```

Создание записи пользователя:

```
INSERT INTO Users (username, email, password_hash, role, is_superuser,
is_staff, is_active, last_login, date_joined)
VALUES ('new_user', 'newuser@email.com', 'hashed_pwd...', 'user', FALSE,
FALSE, TRUE, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Логирование регистрации:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
VALUES (LAST_INSERT_ID(), NULL, 'USER_REGISTERED',
'{"username": "new_user", "email": "newuser@email.com"}',
CURRENT_TIMESTAMP);
```

Количество запросов: 3. Зависимость от объектов в БД: O(1).

Количество задействованных таблиц: 2 (Users, Logs).

Сценарий 2: Загрузка нового изображения

Текст запросов:

Создание публикации:

```
INSERT INTO Publications (user_id, title, description, file, image_format,
width, height, file_size_mb, created_at, updated_at)
VALUES (1, 'Киберпанк город', 'Описание фото', 'media/photos/cyber.png',
'PNG', 1920, 1080, 2.5, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Логирование создания:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
VALUES (1, LAST_INSERT_ID(), 'CREATE', '{"title": "Киберпанк город",
"format": "PNG"}', CURRENT_TIMESTAMP);
```

Количество запросов: 2. Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 2 (Publications, Logs).

Сценарий 3: Редактирование метаданных

Текст запросов:

Обновление записи:

```
UPDATE Publications
SET title = 'Новое название', description = 'Дополненное описание',
updated_at = CURRENT_TIMESTAMP
WHERE id = 101 AND user_id = 1;
```

Логирование обновления:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
VALUES (1, 101, 'UPDATE', '{"image_id": "101", "title": "Новое название",
"description": "Дополненное описание"}', CURRENT_TIMESTAMP);
```

Количество запросов: 2. Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 2 (Publications, Logs).

Сценарий 4: Удаление публикации

Текст запросов:

Удаление записи:

```
DELETE FROM Publications
```

WHERE id = 101 AND user_id = 1;

Логирование удаления:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
VALUES (1, 101, 'DELETE', '{"title": "Киберпанк город", "deleted_format":
"PNG"}', CURRENT_TIMESTAMP);
```

Количество запросов: 2. Зависимость от числа объектов: O(1).

Количество задействованных таблиц: 2 (Publications, Logs).

Сценарий 5: Сложный поиск и фильтрация

Текст запросов:

Поиск по условиям:

```
SELECT id, title, file, image_format, created_at
FROM Publications
WHERE title LIKE '%animals%'
AND image_format = 'JPEG'
AND created_at BETWEEN '2026-01-01 00:00:00' AND '2026-12-31
23:59:59';
```

Логирование поиска:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
VALUES (1, NULL, 'EXPORT', '{"search_query": "animals", "filter_format":
"JPEG", "date_range": ["2026-01-01", "2026-12-31"]}', CURRENT_TIMESTAMP);
```

Количество запросов: 2. Зависимость от числа объектов: O(1).

Количество задействованных таблиц: 2 (Publications, Logs).

Сценарий 6: Просмотр статистики системы (Админ)

Текст запросов:

Агрегация данных: группировка публикаций по дням и подсчет их количества:

```
SELECT
DATE(created_at) AS upload_date,
COUNT(id) AS count
```

FROM Publications

GROUP BY DATE(created_at)

ORDER BY upload_date ASC;

Запись в логи (фиксация факта просмотра статистики):

INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)

VALUES (1, NULL, 'STATS_VIEWED', '{"report_type": "uploads_per_day"}',

CURRENT_TIMESTAMP);

Количество запросов: 2. Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 2 (Publications, Logs).

Сценарий 7: Экспорт базы данных системы (Админ)

Текст запросов: 1, 2, 3. Выгрузка всех данных из таблиц (система выполняет последовательные запросы к каждой таблице для последующей сериализации в единый JSON-файл на стороне бэкенда):

SELECT * FROM Users;

SELECT * FROM Publications;

SELECT * FROM Logs;

Логирование экспорта (фиксация факта создания резервной копии):

INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)

VALUES (1, NULL, 'DATABASE_EXPORTED', '{}',

CURRENT_TIMESTAMP);

Количество запросов: 4 запроса (3 на чтение таблиц + 1 на логирование).

Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 3 (Users, Publications, Logs).

Сценарий 8: Импорт (восстановление) базы данных (Админ)

Текст запросов: 1, 2. Очистка текущих рабочих таблиц (с временным отключением проверок внешних ключей):

SET FOREIGN_KEY_CHECKS = 0;

```
TRUNCATE TABLE Publications;
```

```
TRUNCATE TABLE Users;
```

```
SET FOREIGN_KEY_CHECKS = 1;
```

3, 4. Массовая вставка распарсенных данных из JSON-файла (бэкенд склеивает данные в единые большие запросы):

```
INSERT INTO Users (id, username, email, password_hash, role, is_superuser,  
is_staff, is_active, last_login, date_joined)
```

```
VALUES
```

```
(1, 'admin', 'admin@example.com', 'hash1...', 'admin', TRUE, TRUE, TRUE,  
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
```

```
(2, 'user_2', 'user2@example.com', 'hash2...', 'user', FALSE, FALSE, TRUE,  
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

```
-- (В реальности запрос содержит тысячи строк)
```

```
INSERT INTO Publications (id, user_id, title, description, file, image_format,  
width, height, file_size_mb, created_at, updated_at)
```

```
VALUES
```

```
(101, 1, 'Фото 1', 'Описание', 'file1.png', 'PNG', 1920, 1080, 2.5,  
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
```

```
(102, 2, 'Фото 2', 'Описание', 'file2.jpg', 'JPEG', 800, 600, 1.2,  
CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Логирование импорта (фиксация факта восстановления системы):

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
```

```
VALUES (1, NULL, 'DATABASE_IMPORTED', '{"source_file":  
"db_dump.json"}', CURRENT_TIMESTAMP);
```

Количество запросов: 5 базовых логических запросов (2 на очистку, 2 на вставку данных, 1 на лог) + 2 сервисных запроса на переключение ключей.

Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 3 (Users, Publications, Logs).

Сценарий 9: Изменение пароля в профиле

Текст запросов:

Получение текущего хэша пароля (для проверки бэкендом совпадения со старым паролем, который ввел пользователь):

```
SELECT password_hash
```

```
FROM Users
```

```
WHERE id = 1;
```

Обновление хэша пароля (если старый пароль совпал):

```
UPDATE Users
```

```
SET password_hash = 'new_hashed_password_string...'
```

```
WHERE id = 1;
```

Логирование изменения пароля (фиксация события безопасности):

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
```

```
VALUES (1, NULL, 'PASSWORD_CHANGED', '{}',
```

```
CURRENT_TIMESTAMP);
```

Количество запросов: 3 запроса (1 на чтение, 1 на обновление, 1 на логирование). Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 2 (Users, Logs).

Сценарий 10: Восстановление / Установка нового пароля

Текст запросов:

Обновление хэша пароля (после того как бэкенд успешно провалидировал токен восстановления из URL и извлек из него id пользователя):

```
UPDATE Users
```

```
SET password_hash = 'new_recovered_password_hash...'
```

```
WHERE id = 1;
```

Логирование успешного восстановления:

```
INSERT INTO Logs (user_id, publication_id, action, payload, timestamp)
```

VALUES (1, NULL, 'PASSWORD_RECOVERED', '{}',
CURRENT_TIMESTAMP);

Количество запросов: 2 запроса (1 на обновление, 1 на логирование).

Зависимость от числа объектов: $O(1)$.

Количество задействованных таблиц: 2 (Users, Logs).

1.3.2. Аналог модели данных для SQL СУБД

```
// Коллекция Users
{
  "_id": "ObjectId",
  "username": "String",
  "password_hash": "String",
  "first_name": "String (Null)",
  "last_name": "String (Null)",
  "email": "String",
  "role": "String",
  "is_active": "Boolean",
  "is_staff": "Boolean",
  "is_superuser": "Boolean",
  "last_login": "Date (Null)",
  "date_joined": "Date"
}

// Коллекция Publications
{
  "_id": "ObjectId",
  "author": {
    "id": "ObjectId",
    "username": "String"
  },
  "title": "String",
  "description": "String (Null)",
  "file": {
    "url": "String",
    "format": "String",
    "size_mb": "Double",
    "width": "Int32",
    "height": "Int32"
  },
  "created_at": "Date",
  "updated_at": "Date"
}

// Коллекция Logs
{
  "_id": "ObjectId",
  "user_id": "ObjectId",
  "publication_id": "ObjectId (Null)",
  "action": "String",
  "payload": "Object",
  "timestamp": "Date"
}
```

Рис №3. NoSQL модель

Описание таблиц, типов данных и сущностей

Коллекция users (Пользователи)

Что хранит: Учетные данные, роли и метки времени.

Оценка веса полей (V u):

Базовый оверхед документа: 5 байт

`_id`: $1(\text{тип}) + 4(\text{"_id\0"}) + 12(\text{ObjectId}) = 17$ байт

`username` ($N \approx 10$): $1 + 9(\text{"username\0"}) + 4 + 20 + 1 = 35$ байт

`password_hash` ($N=88$): $1 + 14(\text{"password_hash\0"}) + 4 + 176 + 1 = 196$ байт

`first_name` ($N \approx 10$): $1 + 11(\text{"first_name\0"}) + 4 + 20 + 1 = 37$ байт

`last_name` ($N \approx 10$): $1 + 10(\text{"last_name\0"}) + 4 + 20 + 1 = 36$ байт

`email` ($N \approx 25$): $1 + 6(\text{"email\0"}) + 4 + 50 + 1 = 62$ байт

`role` ($N \approx 5$): $1 + 5(\text{"role\0"}) + 4 + 10 + 1 = 21$ байт

`is_active` (Bool): $1 + 10(\text{"is_active\0"}) + 1 = 12$ байт

`is_staff` (Bool): $1 + 9(\text{"is_staff\0"}) + 1 = 11$ байт

`is_superuser` (Bool): $1 + 13(\text{"is_superuser\0"}) + 1 = 15$ байт

`last_login` (Date): $1 + 11(\text{"last_login\0"}) + 8 = 20$ байт

`date_joined` (Date): $1 + 12(\text{"date_joined\0"}) + 8 = 21$ байт

Средний объем одного документа (`V u m o n g o`): 488 байт.

Коллекция `publications` (Публикации / Изображения)

Что хранит: Метаданные файлов, сгруппированные в поддокумент, и денормализованные данные об авторе для быстрого чтения ленты.

Оценка веса полей (`V p`):

Базовый оверхед: 5 байт

`_id`: 17 байт

`author` (вложенный объект): $1(\text{тип}) + 7(\text{"author\0"}) + (5 \text{ оверхед} + 16 \text{ id} + 35 \text{ username}) = 64$ байта

`title` ($N \approx 30$): $1 + 6(\text{"title\0"}) + 4 + 60 + 1 = 72$ байта

`description` ($N \approx 100$): $1 + 12(\text{"description\0"}) + 4 + 200 + 1 = 218$ байт

`file` (вложенный объект): $1 + 13(\text{"file\0"}) + (5 \text{ оверхед} + 130 \text{ url} + 21 \text{ format} + 17 \text{ size_mb} + 11 \text{ width} + 12 \text{ height}) = 210$ байт

`created_at` (Date): $1 + 11(\text{"created_at\0"}) + 8 = 20$ байт

`updated_at` (Date): $1 + 11(\text{"updated_at\0"}) + 8 = 20$ байт

Средний объем одного документа (`V p m o n g o`): 626 байт.

Коллекция logs (Журнал событий)

Что хранит: Хронологию действий пользователей.

Оценка веса полей (V_l):

Базовый оверхед: 5 байт

$_id$: 17 байт

$user_id$: $1 + 8 + 12 = 21$ байт

$publication_id$: $1 + 15 + 12 = 28$ байт

$action$ ($N=6$): $1 + 7 + 4 + 12 + 1 = 25$ байт

$payload$ (вложенный объект $N \approx 50$ данных): $1 + 8("payload\0") + (5 \text{ байт оверхед} + \sim 114 \text{ байт данных}) \approx 128$ байт

$timestamp$ (Date): $1 + 10 + 8 = 19$ байт

Средний объем одного документа ($V_{l m o n g o}$): 243 байта.

Оценка объема информации, хранимой в модели

Пусть N — общее количество загруженных публикаций. Как и в реляционной модели, примем:

Количество пользователей: $N_u = 0.2 \cdot N$ Количество логов: $N_l = 10 \cdot N$

Объем данных для хранения базы из N публикаций вычисляется по формуле:

$$\begin{aligned} V(N) &= V_{u m o n g o} \cdot N_u + V_{p m o n g o} \cdot N + V_{l m o n g o} \cdot N_l \\ &= 488 \cdot (0.2 \cdot N) + 626 \cdot N + 243 \cdot (10 \cdot N) \\ V(N) &= 97.6 \cdot N + 626 \cdot N + 2430 \cdot N \\ V(N) &= 3153.6 \cdot N \end{aligned}$$

Избыточность данных

Чистый объем данных (без учета технических ключей, вложенности и BSON оверхеда на строковые ключи полей). Возьмем "чистые" значения полезной нагрузки из расчета реляционной модели:

$$V_{p u r e}(N) = 1641.8 \cdot N \text{ (байт)}$$

Избыточность (R) вычисляется как отношение фактического объема (V_{actual}) к чистому (V_{pure}):

$$R = (3153.6 * N) / (1641.8 * N) \approx 1.921$$

Вывод: Избыточность нереляционной модели составляет 92.1%. Это значительно выше SQL-аналога за счет хранения ключей в каждом документе и применения денормализации (дублирования поля username).

Итоговая формула зависимости: $V(N) \approx 3154 \cdot N$ (байт)

Направление роста модели при увеличении количества объектов каждой сущности

Рост модели происходит линейно, но с высокими коэффициентами из-за структуры BSON. При увеличении числа объектов одной сущности (и зафиксированном объеме остальных), размер увеличивается по формулам:

Через пользователей (Users): $modelSizeUsers(x) = 488 \cdot x + C$

Через публикации (Publications): $modelSizePublications(x) = 626 \cdot x + C$

Через логи (Logs): $modelSizeLogs(x) = 243 \cdot x + C$

Комплексный рост (когда на 1 пользователя приходится 5 публикаций и 50 логов):

$$modelSizeTotal(x) = 488 \cdot x + 626 \cdot (5 \cdot x) + 243 \cdot (50 \cdot x) \\ modelSizeTotal(x) = 488 \cdot x + 3130 \cdot x + 12150 \cdot x = 15768 \cdot x$$

(байт на каждого нового пользователя с его экосистемой).

Примеры данных

Документ users:

```
{
  "_id": {"$oid": "64f1a2b3e4b0a1b2c3d4e5f6"},
  "username": "art_lover",
  "first_name": "Анна",
  "last_name": "Смирнова",
  "email": "anna.art@example.com",
```

```
"password_hash": "pbkdf2_sha256$260000$XyZ123HashString...",
"role": "user",
"is_active": true,
"is_staff": false,
"is_superuser": false,
"last_login": {"$date": "2026-04-05T15:24:00Z"},
"date_joined": {"$date": "2026-04-05T15:24:00Z"}
}
```

Документ publications:

```
{
  "_id": {"$oid": "75f2b3c4d5e6f7a8b9c0d1e2"},
  "author": {
    "id": {"$oid": "64f1a2b3e4b0a1b2c3d4e5f6"},
    "username": "art_lover"
  },
  "title": "Киберпанк город",
  "description": "Концепт-арт ночного города с неоновыми вывесками.",
  "file": {
    "url": "media/photos/2026/4/cyber_city_01.png",
    "format": "PNG",
    "size_mb": 8.4,
    "width": 3840,
    "height": 2160
  },
  "created_at": {"$date": "2026-04-05T15:24:00Z"},
  "updated_at": {"$date": "2026-04-05T15:24:00Z"}
}
```

Документ logs:

```
{
  "_id": {"$oid": "86f3c4d5e6f7a8b9c0d1e2f3"},
  "user_id": {"$oid": "64f1a2b3e4b0a1b2c3d4e5f6"},
  "publication_id": {"$oid": "75f2b3c4d5e6f7a8b9c0d1e2"},
  "action": "CREATE",
  "payload": {
    "title": "Киберпанк город",
    "format": "PNG"
  },
  "timestamp": {"$date": "2026-04-05T15:24:00Z"}
}
```

Примеры запросов

Сценарий 1: Регистрация нового пользователя

Количество запросов: 3

Количество задействованных коллекций: 2 (users, logs)

1. Проверка на существование email (Ошибка валидации)

```
db.users.find_one(
  {"email": "newuser@email.com"},
  {"_id": 1}
)
```

2. Создание аккаунта

```
db.users.insert_one({
  "username": "new_user",
  "email": "newuser@email.com",
  "password_hash": "hashed_password_string...",
  "role": "user",
  "is_active": True,
```

```
"is_staff": False,  
"is_superuser": False,  
"date_joined": datetime.datetime.utcnow(),  
"last_login": datetime.datetime.utcnow()  
})
```

3. Запись в логи

```
db.logs.insert_one({  
"user_id": ObjectId("ID_НОВОГО_ПОЛЬЗОВАТЕЛЯ"),  
"publication_id": None,  
"action": "USER_REGISTERED",  
"payload": {"email": "newuser@email.com"},  
"timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 2: Загрузка нового изображения / Create

Количество запросов: 2

Количество задействованных коллекций: 2 (publications, logs)

1. Создание публикации с вложенными метаданными файла

```
db.publications.insert_one({  
"author": {  
"id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),  
"username": "new_user"  
},  
"title": "Мое первое фото",  
"description": "Описание фотографии...",  
"file": {  
"url": "media/uploads/2026/photo.png",  
"format": "PNG",
```



```
"size_mb": 8.5,  
"width": 1920,  
"height": 1080  
},  
"created_at": datetime.datetime.utcnow(),  
"updated_at": datetime.datetime.utcnow()  
}))
```

2. Запись в логи

```
db.logs.insert_one({  
"user_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),  
"publication_id": ObjectId("ID_НОВОЙ_ПУБЛИКАЦИИ"),  
"action": "image_uploaded",  
"payload": {"title": "Мое первое фото", "format": "PNG"},  
"timestamp": datetime.datetime.utcnow()  
}))
```

Сценарий 3: Редактирование метаданных публикации / Update

Количество запросов: 2

Количество задействованных коллекций: 2 (publications, logs)

1. Обновление текстовых полей (проверка author.id гарантирует, что редактор — автор)

```
db.publications.update_one(  
{  
"_id": ObjectId("ID_ПУБЛИКАЦИИ"),  
"author.id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ")  
},  
{  
"$set": {
```

```

    "title": "Новое название",
    "description": "Дополненное описание",
    "updated_at": datetime.datetime.utcnow()
}
}
)

```

2. Запись в логи

```

db.logs.insert_one({
    "user_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),
    "publication_id": ObjectId("ID_ПУБЛИКАЦИИ"),
    "action": "metadata_updated",
    "payload": {"changed_fields": ["title", "description"]},
    "timestamp": datetime.datetime.utcnow()
})

```

Сценарий 4: Удаление публикации / Delete

Количество запросов: 2

Количество задействованных коллекций: 2 (publications, logs)

1. Удаление документа

```

db.publications.delete_one({
    "_id": ObjectId("ID_ПУБЛИКАЦИИ"),
    "author.id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ")
})

```

2. Запись в логи

```

db.logs.insert_one({
    "user_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),
    "publication_id": ObjectId("ID_ПУБЛИКАЦИИ"),

```

```
"action": "image_deleted",  
"payload": None,  
"timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 5: Сложный поиск и фильтрация / Read & Search

Количество запросов: 2

Количество задействованных коллекций: 2 (publications, logs)

1. Поиск по регулярному выражению (название), точному совпадению (формат) и диапазону (дата)

```
db.publications.find({  
  "title": {"$regex": "animals", "$options": "i"},  
  "file.format": "JPEG",  
  "created_at": {  
    "$gte": datetime.datetime(2026, 1, 1, 0, 0, 0),  
    "$lte": datetime.datetime(2026, 12, 31, 23, 59, 59)  
  }  
})
```

2. Запись факта поиска в логи (для аналитики)

```
db.logs.insert_one({  
  "user_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),  
  "publication_id": None,  
  "action": "search_executed",  
  "payload": {"query": "animals", "format": "JPEG"},  
  "timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 6 (Админ): Просмотр статистики системы

Количество запросов: 2

Количество задействованных коллекций: 2 (publications, logs)

1. Агрегация данных: группировка публикаций по дням и подсчет их количества

```
db.publications.aggregate([
  {
    "$group": {
      "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$created_at"}},
      "count": {"$sum": 1}
    }
  },
  {
    "$sort": {"_id": 1} # Сортировка по дате по возрастанию
  }
])
```

2. Запись в логи

```
db.logs.insert_one({
  "user_id": ObjectId("ID_АДМИНИСТРАТОРА"),
  "action": "stats_viewed",
  "payload": {"report_type": "uploads_per_day"},
  "timestamp": datetime.datetime.utcnow()
})
```

Сценарий 7: Экспорт базы данных системы

Количество запросов: 4 (на уровне БД)

Количество задействованных коллекций: 3 (users, publications, logs)

1, 2, 3. Выгрузка всех данных (оборачиваем курсор в list для сериализации)

```
users_data = list(db.users.find({}))  
publications_data = list(db.publications.find({}))  
logs_data = list(db.logs.find({}))
```

4. Логирование экспорта

```
db.logs.insert_one({  
    "user_id": ObjectId("ID_АДМИНИСТРАТОРА"),  
    "action": "database_exported",  
    "payload": None,  
    "timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 8: Импорт (восстановление) базы данных

Количество запросов: 5

Количество задействованных коллекций: 3 (users, publications, logs)

1, 2. Полная очистка текущих рабочих коллекций

```
db.users.delete_many({})  
db.publications.delete_many({})
```

3, 4. Массовая вставка распарсенных данных из JSON-файла

```
db.users.insert_many(parsed_users_list)  
db.publications.insert_many(parsed_publications_list)
```

5. Запись в логи о факте восстановления

```
db.logs.insert_one({  
    "user_id": ObjectId("ID_АДМИНИСТРАТОРА"),  
    "action": "database_imported",
```

```
"payload": {"source_file": "db_dump.json"},  
"timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 9: Изменение пароля в профиле

Количество запросов: 3

Количество задействованных коллекций: 2 (users, logs)

1. Бэкенд запрашивает хеш текущего пароля пользователя

```
db.users.find_one(  
{"_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ")},  
{"password_hash": 1}  
)
```

2. Если пароль совпал, обновляем хэш на новый

```
db.users.update_one(  
{"_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ")},  
{"$set": {"password_hash": "new_hashed_password_string..."}}  
)
```

3. Запись в логи

```
db.logs.insert_one({  
"user_id": ObjectId("ID_ТЕКУЩЕГО_ПОЛЬЗОВАТЕЛЯ"),  
"action": "password_changed",  
"payload": None,  
"timestamp": datetime.datetime.utcnow()  
})
```

Сценарий 10: Восстановление / Установка нового пароля

Количество запросов: 2

Количество задействованных коллекций: 2 (users, logs)

1. Обновление пароля по ID пользователя (бэкенд сам расшифровывает JWT/токен из URL)

```
db.users.update_one(  
    {"_id": ObjectId("ID_ПОЛЬЗОВАТЕЛЯ_ИЗ_ТОКЕНА")},  
    {"$set": {"password_hash": "new_recovered_password_hash..."}}  
)
```

2. Запись в логи

```
db.logs.insert_one({  
    "user_id": ObjectId("ID_ПОЛЬЗОВАТЕЛЯ_ИЗ_ТОКЕНА"),  
    "action": "password_recovered",  
    "payload": None,  
    "timestamp": datetime.datetime.utcnow()  
})
```

1.3.3 Сравнение моделей

Удельный объем информации

Анализ обеих моделей показывает существенную разницу в потреблении дискового пространства:

Реляционная модель (SQL): Объем данных зависит от количества публикаций по формуле $V(N) \approx 1771 \cdot N$ байт. Избыточность составляет всего ~7.8%, так как структура строго нормализована, а связи реализуются через компактные внешние ключи (Foreign Keys типа INT, занимающие 4 байта).

Нереляционная модель (MongoDB): Объем данных вычисляется по формуле $V(N) \approx 3154 \cdot N$ байт. Избыточность достигает ~92.1%.

Почему нереляционная модель занимает почти в 2 раза больше места?

Специфика формата BSON: В каждом документе MongoDB хранятся не только сами значения, но и строковые названия ключей (например, строка "username" повторяется в каждом документе коллекции users).

Типы данных: Идентификаторы ObjectId занимают 12 байт (против 4 байт у INT в SQL). Базовый оверхед каждого документа также "съедает" дополнительную память.

Денормализация: Для ускорения чтения ленты публикаций в MongoDB была применена денормализация (внедрение имени автора username прямо в документ publications). Это избавляет от необходимости делать \$lookup (аналог JOIN), но увеличивает дублирование данных.

Запросы по отдельным юзкейсам

Поскольку в системе реализовано обязательное сквозное логирование (запись в отдельную таблицу/коллекцию Logs при любом действии), базовое количество запросов к БД (на уровне приложения) для выполнения бизнес-логики совпадает в обеих моделях. Однако сложность их выполнения под капотом отличается.

Сценарий	SQL модель	NoSQL модель	Сравнение и комментарий
1. Регистрация пользователя	3 запроса / таблицы	3 запроса / 2 коллекции	Идентично. Валидация + вставка юзера + лог.
2. Загрузка изображения	2 запроса / таблицы	2 запроса / 2 коллекции	В SQL данные вставляются линейно. В NoSQL формируется вложенный документ (метаданные файла).
3. Редактирование метаданных	2 запроса / таблицы	2 запроса / 2 коллекции	Идентично. Осуществляется UPDATE по ID публикации.

4. Удаление публикации	2 запроса / 2 таблицы	2 запроса / 2 коллекции	Идентично.
5. Поиск и фильтрация	2 запроса / 2 таблицы	2 запроса / 2 коллекции	В NoSQL поиск отработает эффективнее (если нужно вывести имя автора), так как оно денормализовано. В SQL для вывода имени автора вместе с картинкой потребовался бы JOIN с таблицей Users.
6. Просмотр статистики	2 запроса / 2 таблицы	2 запроса / 2 коллекции	SQL использует нативный GROUP BY DATE(). MongoDB использует Aggregation Framework, что синтаксически сложнее, но работает сопоставимо.
7. Экспорт БД	4 запроса / 3 таблицы	4 запроса / 3 коллекции	В обоих случаях выполняется полный SELECT * / find({}) по всем сущностям.
8. Импорт БД	5+ запросов / 3 таблицы	5 запросов / 3 коллекции	В SQL процесс сложнее: необходимо временно отключать проверку внешних ключей (FOREIGN_KEY_CHECKS = 0), чтобы избежать конфликтов при вставке.
9. Изменение пароля	3 запроса / 2 таблицы	3 запроса / 2 коллекции	Идентично.
10. Восстановление пароля	2 запроса / 2 таблицы	2 запроса / 2 коллекции	Идентично.

Вывод - что лучше, SQL или NoSQL модель

Для реализации backend-части системы галереи изображений

Реляционная модель (SQL) является более предпочтительной.

Обоснование выбора:

Экономия ресурсов: Реляционная БД требует почти в 2 раза меньше дискового пространства ($1771 \cdot N$ против $3154 \cdot N$ байт). При масштабировании системы до миллионов публикаций эта разница станет критичной с точки зрения стоимости серверов.

Структура данных: Сущности в данной системе имеют жесткую, заранее известную схему. У пользователя всегда есть логин и пароль, у картинки — формат, ширина и высота, а у лога — время и тип действия. Отсутствие "плавающей" схемы сводит на нет главное преимущество NoSQL.

Связанность данных (ACID): При удалении пользователя реляционная БД позволяет автоматически каскадно удалить все его публикации и логи (через ON DELETE CASCADE), гарантируя целостность данных на уровне самой СУБД. В MongoDB эту консистентность придется поддерживать на уровне кода приложения.

Когда стоило бы выбрать NoSQL (MongoDB)? Выбор в пользу MongoDB был бы оправдан только в том случае, если бы метаданные картинок были сильно неструктурированными. Например, если бы пользователи загружали фотографии с огромным набором различных и непредсказуемых EXIF-данных (настройки камеры, геолокация, линзы, вспышка), которые было бы неудобно хранить в жестких столбцах SQL-таблицы, а удобно складывать в виде гибкого JSON-объекта payload. В рамках текущей задачи такая гибкость не требуется.

1.4. Разработанное приложение

Разработанное приложение представляет собой современный полнофункциональный веб-сервис, построенный на клиент-серверной архитектуре с разделением на микросервисы, работающие в изолированных контейнерах. Бэкенд-часть написана на Python (версия 3.14 и выше) с использованием мощного фреймворка Django (6.0.3 и выше) и Django REST Framework (DRF) для создания высокопроизводительного RESTful API. В

качестве WSGI-сервера используется Gunicorn, а для обработки изображений задействована библиотека Pillow. Фронтенд функционирует как Single Page Application (SPA), разработанный на Vue.js (версия 3) с TypeScript. Сборка проекта осуществляется с помощью Vite, управление состоянием — через Pinia, а стилизация интерфейса — с помощью Tailwind CSS. Все компоненты, включая базу данных MongoDB (образ mongo:7.0, интегрирован с Django через django-mongodb-backend), бэкенд, фронтенд и веб-сервер Nginx, выступающий в роли обратного прокси, контейнеризированы с помощью Docker и Docker Compose.

Приложение предоставляет пользователям широкий набор функций. Основной функционал включает Загрузку изображений, при которой система автоматически определяет формат файла, его размер в мегабайтах, а также ширину и высоту. Для удобства работы с контентом реализованы Поиск и фильтрация по текстовым полям (название и описание) с применением SearchFilter и DjangoFilterBackend, а также возможность фильтровать результаты по лайкам текущего пользователя (liked_by_me). Редактирование доступно только авторам изображений и ограничено полями названия и описания. В проекте реализованы Социальные функции в виде системы лайков с подсчетом общего количества.

В приложении внедрена комплексная система событийно-ориентированного аудита (логирования), обеспечивающая полную прозрачность всех ключевых процессов. В базу данных MongoDB автоматически записываются 17 типов системных событий, разделенных по функциональным областям.

Для обеспечения сохранности данных и удобства администрирования разработана подсистема массового резервного копирования и восстановления. При экспорте система формирует единый .zip-архив, содержащий полный JSON-дамп коллекций MongoDB и структуру директорий медиафайлов. Процесс импорта обладает высокой отказоустойчивостью и построен по стратегии безопасного отката: перед накатыванием бэкапа и сбросом базы

создается теньевая копия текущих изображений и БД, которая мгновенно возвращается на место при возникновении любых сбоев. Входной архив проходит валидацию на уязвимости (подмены путей) и превышение лимитов распаковки. Операции импорта и экспорта также регистрируются в системе логов с указанием объемов данных.

Система прав доступа настроена на основе классов разрешений Django. Просмотр данных доступен всем пользователям, включая неавторизованных (благодаря классу `IsAuthenticatedOrReadOnly`). Управление контентом (редактирование и удаление) возможно только для автора публикации или администратора сайта, что обеспечивается кастомным классом `IsAuthorOrAdminOrReadOnly`. Администраторы также могут полностью ограничить право публикации для любого пользователя, установив флаг `publish_blocked`. Пользователи в системе разделены на две роли: `user` и `admin`.

Особое внимание уделено Безопасности и фильтрации файлов. На уровне создания публикаций действует механизм троттлинга (`Anti-Spam`), ограничивающий обычных пользователей до 5 публикаций в час и 30 в день. В части фильтрации файлов используется стандартный класс Django `ImageField`, который с помощью библиотеки `Pillow` проверяет валидность загружаемого файла как изображения. Тем не менее, явные кастомные проверки на сложные вредоносные вложения не реализованы, и не предусмотрены жесткие лимиты на максимальный размер загружаемого файла и строгие ограничения (белый список) форматов на уровне моделей.

Развертывание проекта описано декларативно с использованием `Docker Compose`. Конфигурация `docker-compose.yml` запускает четыре взаимозависимых сервиса: `MongoDB`, бэкенд, фронтенд (в режиме разработчика через `Vite`) и `Nginx`. Конфиденциальные настройки передаются через переменные окружения из файла `.env`. При первоначальном старте контейнера бэкенда автоматически выполняются миграции, создаются пользователи и загружаются стандартные изображения по умолчанию.

Изображения приложения:

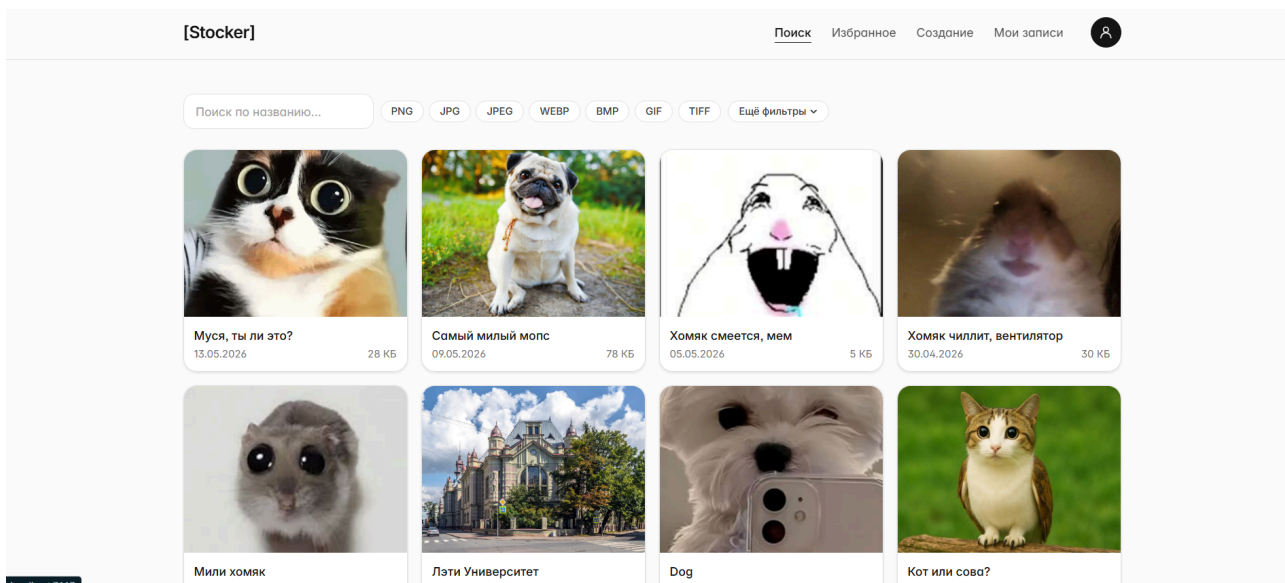


Рис №4. Главная страница

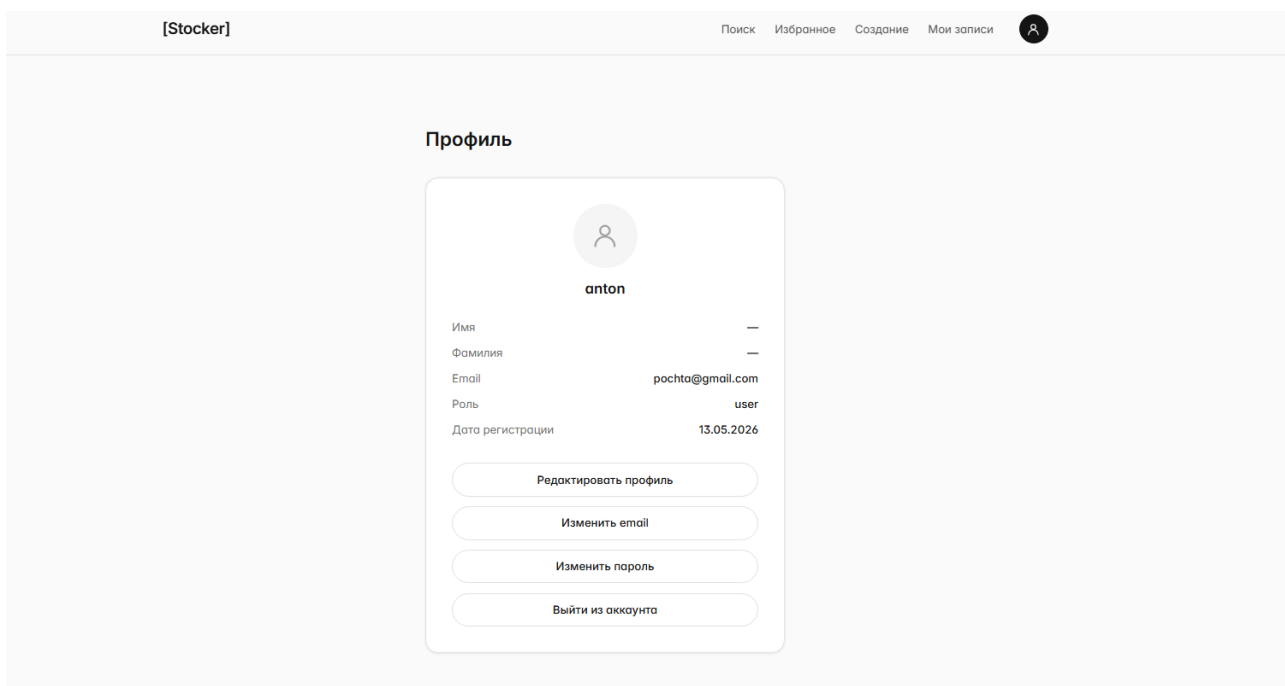


Рис №5. Профиль пользователя

ЗАКЛЮЧЕНИЕ

Достигнутые результаты

В ходе выполнения курсовой работы была успешно решена поставленная задача по проектированию и разработке масштабируемого веб-приложения «Хранилище изображений с CRUD». Проект реализован на современном стеке технологий, включая Python/Django/DRF для бэкенда и Vue.js 3/TypeScript для фронтенда.

Ключевые достигнутые результаты включают: полнофункциональный CRUD для графического контента; эффективную систему поиска и фильтрации; многоуровневую авторизацию и сквозное логирование; а также контейнеризацию всех компонентов с помощью Docker Compose для упрощения развертывания.

Недостатки и пути для улучшения полученного решения

В процессе работы были выявлены несколько областей для оптимизации. Детальный анализ показал, что для данной системы **реляционная модель (SQL)** была бы более предпочтительна, так как выбранная MongoDB продемонстрировала высокую избыточность (92.1%). Также стоит рассмотреть S3 хранилища для хранения изображений. Большинство изображений сейчас могут весить довольно много, поэтому также стоит рассмотреть сжатие, для того чтобы страница загружалась быстрее, но добавить кнопку для скачивания в полном качестве.

Будущее развитие решения

Дальнейшее развитие проекта должно быть сфокусировано на переходе к реляционной СУБД для обеспечения лучшей целостности данных, усилении безопасности через строгую валидацию загрузок и реализации расширенного поиска по всем метаданным. Планируется доработка административного интерфейса в части статистики и функций массового экспорта/импорта данных.

ПРИЛОЖЕНИЕ А

ДОКУМЕНТАЦИЯ ПО СБОРКЕ И РАЗВЕРТЫВАНИЮ ПРИЛОЖЕНИЯ

Для развертывания проекта необходимо клонировать репозиторий проекта.

```
git clone git@github.com:moevm/nsql1h26-img.git
```

Далее перейти в папку клонированного репозитория и запустить docker compose файл:

```
docker compose build --no-cache && docker compose up -d
```

После этого сайт будет доступен на <http://localhost:2026>

ПРИЛОЖЕНИЕ Б

ИНСТРУКЦИЯ ДЛЯ ПОЛЬЗОВАТЕЛЯ

Просмотр изображений:

При переходе на сайт, вы можете сразу просматривать и искать по изображениям, смотреть количество лайков на изображении, кто его опубликов, просматривать профиль, того кто опубликов.

Регистрация аккаунта:

Если же вы хотите публиковать изображения, то вам необходимо зарегистрироваться. Для этого нажмите на иконку человека справа сверху сайта в меню. После этого под формой нажать на “Зарегистрироваться”. Далее ввести данные необходимые для регистрации. После чего вы будете авторизованы в аккаунт (если этого не произошло, то кликните еще раз по иконке человека и авторизуйтесь).

Теперь вы полноценный пользователь, который может публиковать изображения (не более 5 в час и не более 30 в день). Для увеличения лимитов, необходимо обратиться к администратору.

Публикация изображений:

Перейти в меню в “Создание”, после чего загрузить изображение кликом по полю для изображения или переносом его в прямоугольник для загрузки. Далее необходимо ввести название и по желанию - описание. После чего нажать на кнопку “Опубликовать”. После этого запись появится на главной странице и в разделе “Мои записи”

Редактирование изображений:

Необходимо перейти на изображение, которое вы хотите отредактировать, это можно сделать найдя данную публикацию через поиск или через “Мои записи”. После перехода на страницу изображения (кликом по нему) нужно

нажать на кнопку “Редактировать”. Внести изменения в название и\или описание и нажать на кнопку “Сохранить”.

Удаление изображений:

Необходимо перейти на изображение, которое вы хотите удалить, это можно сделать найдя данную публикацию через поиск или через “Мои записи”. После перехода на страницу изображения (кликом по нему) нужно нажать на кнопку “Удалить”. Должно появиться всплывающее окно, в котором можно нажать “Отменить” для отмены или “Удалить” для подтверждения.

Изменение e-mail:

Для начала вам необходимо перейти в свой профиль. Для этого вы должны быть авторизованы и нужно нажать по иконке пользователя в меню (справа сверху). После чего нажать на изменить e-mail. После этого введите e-mail и нажать “Отправить код”, на ваш НОВЫЙ email вам придет код подтверждения который нужно будет ввести в поле “Код подтверждения” и нажать кнопку “Подтвердить” для подтверждения.

Изменение пароля:

Для начала вам необходимо перейти в свой профиль. Для этого вы должны быть авторизованы и нужно нажать по иконке пользователя в меню (справа сверху). После чего нажать на изменить пароль. После этого введите старый пароль и 2 раза новый пароль в соответствующие поля. После чего нажать на “Сохранить”

Изменение информация профиля/логина:

Для начала вам необходимо перейти в свой профиль. Для этого вы должны быть авторизованы и нужно нажать по иконке пользователя в меню (справа сверху). После чего нажать на “Редактировать профиль”, где вам нужно внести изменение в желаемое поле, после чего нажать на “Сохранить”.